

Backpropagation in a CNN, Part 1: The Setup and the Last Layer

Contents

1. Overview	2
2. Prerequisites	3
3. A small CNN	3
3.1 The architecture and the shapes	3
3.2 The trainable parameters	4
3.3 The loss	5
4. Forward propagation	5
5. What we need: four derivatives	5
5.1 Two parts: a CNN and an ANN	6
5.2 The chains	6
6. The ANN part: $\partial L / \partial W_2$ and $\partial L / \partial b_2$	8
6.1 The first two factors	8
6.2 The last factors	8
6.3 Checking the shapes	9
7. A batch of images	9
8. Summary	11
9. Sources	11
10. Key terms	11

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Backpropagation ([Note 1019](#)); Convolution operation and feature maps ([Note 1042](#)).

1. Overview

Key point: A CNN is trained exactly like an ANN: forward propagation, a loss, then gradient descent on every weight, with the gradients found by the chain rule. We split a small CNN into its CNN part and its ANN part; the ANN part's gradients are the familiar ones, $\partial L / \partial W_2 = (a_2 - y) F^\top$ and $\partial L / \partial b_2 = a_2 - y$.

Backpropagation (G-247) for an ANN was taught in the [backpropagation Notes](#). A CNN adds three new operations, and the **chain rule** (G-371) must pass through each of them:

- convolution;
- max pooling;
- flatten. This Note sets the problem up on the smallest possible CNN and finds the gradients of its last layer. The [part 2 Note](#) goes back through flatten, max pooling and convolution.

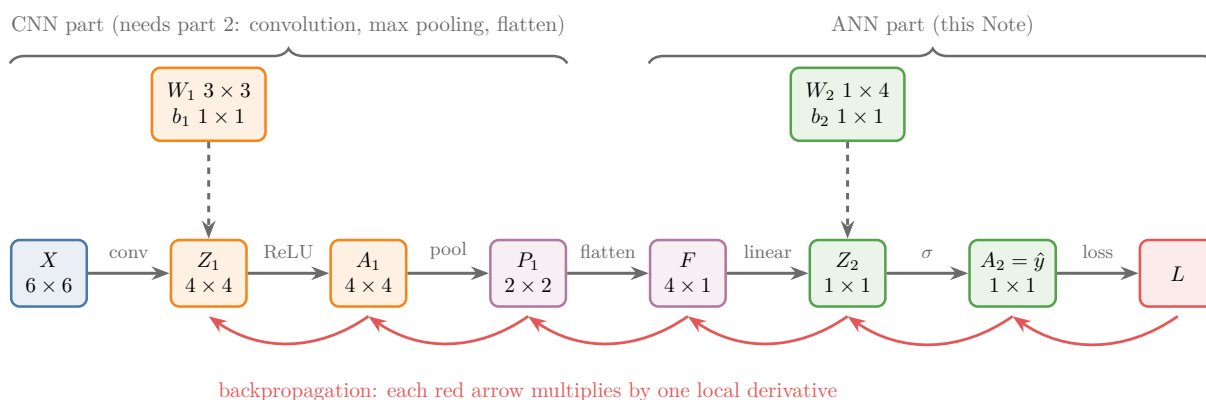


Figure 1: The small CNN as a chain of operations, with the shape of every tensor. The trainable parameters are W_1, b_1 (the filter) and W_2, b_2 (the output node). Backpropagation walks the chain from right to left

In practice Keras computes all of this for us. Knowing how the gradients flow through a CNN still helps us understand and use the tools better.

2. Prerequisites

- The [backpropagation Notes](#), [how it runs](#) and [why it works](#): the chain rule along a network, and the update $w \leftarrow w - \eta \partial L / \partial w$.
- The [CNN vs ANN Note](#): a **filter** (G-778) works like a node, its values are weights.
- The [convolution operation Note](#) and the [pooling Note](#).
- The [log loss Note](#) and the [sigmoid derivative Note](#).

3. A small CNN

Key point: 6×6 image $\rightarrow$ one 3×3 filter with bias $\rightarrow$ ReLU $\rightarrow 2 \times 2$ max pooling $\rightarrow$ flatten $\rightarrow$ one sigmoid node. Shapes: $6 \times 6 \rightarrow 4 \times 4 \rightarrow 4 \times 4 \rightarrow 2 \times 2 \rightarrow 4 \rightarrow 1$.

3.1 The architecture and the shapes

Key point: Convolution gives 4×4 , ReLU keeps the shape, pooling halves it to 2×2 , flatten gives 4 numbers, the output node gives 1.

The network (Figure 1) has:

1. **Input:** a 6×6 greyscale image X .
2. **Convolution:** one 3×3 filter with its bias, giving a 4×4 **feature map** (G-766) ($6 - 3 + 1 = 4$).
3. **ReLU:** negatives become 0; the shape stays 4×4 .
4. **Max pooling:** 2×2 window, stride 2, giving 2×2 .
5. **Flatten:** 4 numbers.
6. **Output:** a single node with sigmoid, giving the prediction $\hat{y}$, a number between 0 and 1.

The network is a binary classifier: is this a picture of a cat or a dog, or, in the Notebook, is this MNIST digit a 1 ($y = 1$) or a 0 ($y = 0$)? The Notebook shrinks real MNIST digits to 6×6 so that every shape matches this Note.

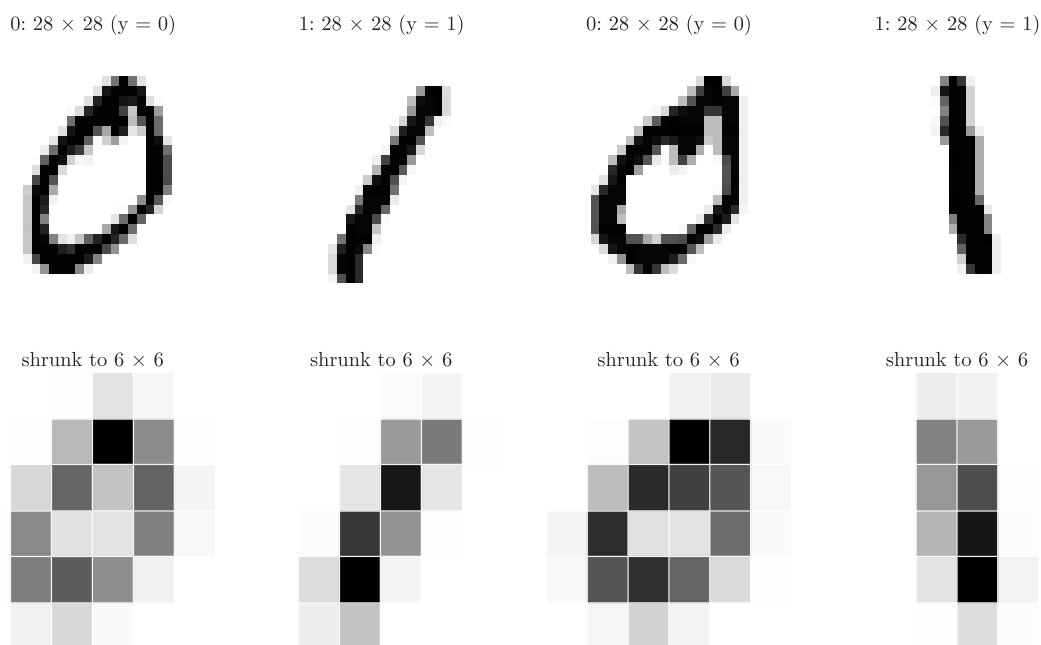


Figure 2: Four of the Notebook’s MNIST digits at their full 28×28 size (top) and shrunk to 6×6 (bottom), the input size of the small CNN. The first 0 is the image used in section 6.

In Figure 2, the 6×6 versions keep only the rough shape, a ring for a 0 and a vertical stroke for a 1.

3.2 The trainable parameters

Key point: 9 filter weights + 1 filter bias + 4 output weights + 1 output bias = 15 parameters.

Only two places in the network hold parameters:

Parameter	Where	Shape	Count
W_1	the filter	3×3	9
b_1	the filter’s bias	1×1	1
W_2	weights of the output node	1×4	4
b_2	bias of the output node	1×1	1

The total is $9 + 1 + 4 + 1 = 15$ **trainable parameters** (G-1999). ReLU, max pooling and flatten have none. Training means finding the 15 values that make the loss smallest.

3.3 The loss

Key point: Binary cross-entropy (log loss), the same loss as in logistic regression.

For one image with target y and prediction $a_2 = \hat{y}$, the loss is the **binary cross-entropy** (G-304) (see the [log loss Note](#)):

$$L = -y \log a_2 - (1 - y) \log(1 - a_2)$$

For a batch of m images the loss is the average of the m single losses. For several classes we would use softmax and categorical cross-entropy instead (see the [loss functions Note](#)). The loss of a CNN is exactly the loss of an ANN.

4. Forward propagation

Key point: Forward propagation (G-797) computes the prediction from the image, one operation at a time: $Z_1 = X * W_1 + b_1$, $A_1 = \text{ReLU}(Z_1)$, $P_1 = \text{maxpool}(A_1)$, $F = \text{flatten}(P_1)$, $Z_2 = W_2 F + b_2$, $A_2 = \sigma(Z_2)$.

Figure 1 is the **logical diagram** (G-1118) of the network: each arrow is one operation, each box one tensor. Written as equations:

Step	Equation	Shape
Convolution	$Z_1 = X * W_1 + b_1$	4×4 (the feature map)
ReLU	$A_1 = \text{ReLU}(Z_1)$	4×4
Max pooling	$P_1 = \text{maxpool}(A_1)$	2×2
Flatten	$F = \text{flatten}(P_1)$	4×1
Output node	$Z_2 = W_2 F + b_2$	$(1 \times 4)(4 \times 1) = 1 \times 1$
Sigmoid	$A_2 = \sigma(Z_2)$	1×1 (the prediction)

Here $*$ is the convolution and b_1 is added to every cell of the feature map. The product is written $W_2 F$, not FW_2 , so that the shapes fit: (1×4) times (4×1) gives 1×1 . With these equations we can code the forward pass and predict for any image; the Notebook does it in NumPy.

5. What we need: four derivatives

Key point: Gradient descent needs $\partial L / \partial W_1$, $\partial L / \partial b_1$, $\partial L / \partial W_2$ and $\partial L / \partial b_2$. Each derivative has the shape of its parameter.

Training starts from random values of W_1, b_1, W_2, b_2 and repeats gradient descent until the loss is small (see the [backpropagation why Note](#)):

$$W_1 \leftarrow W_1 - \eta \frac{\partial L}{\partial W_1}, \quad b_1 \leftarrow b_1 - \eta \frac{\partial L}{\partial b_1}, \quad W_2 \leftarrow W_2 - \eta \frac{\partial L}{\partial W_2}, \quad b_2 \leftarrow b_2 - \eta \frac{\partial L}{\partial b_2}$$

W_1 and W_2 are matrices, so their derivatives are matrices of the same shape: one partial derivative per weight (see the [Jacobian and matrix gradients Note](#)). So the whole task is to find these four derivatives.

5.1 Two parts: a CNN and an ANN

Key point: Everything up to flatten is the CNN part; everything after is an ordinary one-node ANN. Study them separately, then join them.

It helps to see the network as two networks joined together (Figure 1): a **CNN part** (convolution, ReLU, max pooling, flatten) and an **ANN part** (the output node). This Note finds the ANN part's derivatives; the [part 2 Note](#) finds the CNN part's.

5.2 The chains

Key point: W_2 reaches the loss through 3 links; W_1 through 7. The chain rule multiplies the derivative of every link.

A derivative such as $\partial L / \partial W_2$ asks: if W_2 changes by a little, how much does the loss change? W_2 is not connected to L directly. A change in W_2 changes Z_2 , which changes A_2 , which changes L . The chain rule multiplies the three links (see section 5.3 of the [derivatives Note](#)):

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}, \quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2}$$

Only the last factor differs between the two.

For the filter the path is much longer. A change in W_1 changes Z_1 , then A_1 , P_1 , F , Z_2 , A_2 and finally L :

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial F} \cdot \frac{\partial F}{\partial P_1} \cdot \frac{\partial P_1}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial W_1}$$

and $\partial L / \partial b_1$ is the same chain with $\partial Z_1 / \partial b_1$ as its last factor.

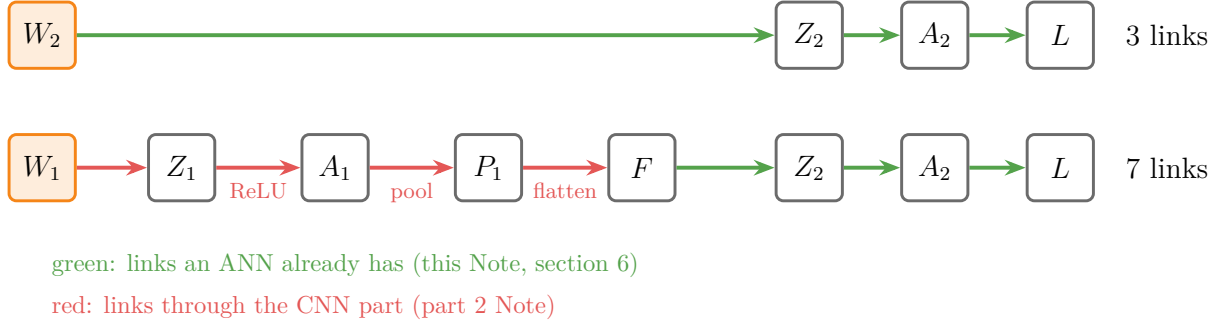


Figure 3: The two paths from a parameter to the loss. Each arrow is one link of the chain rule. Green links are those of an ordinary ANN; red links pass through the CNN part.

Figure 3 shows the two chains side by side: the last three links are shared, so the work of section 6 is reused for the filter; the four red links, through the CNN part, are left for the part 2 Note.

Figure 4 plays the whole plan. First the forward chain is drawn, one operation per arrow. Then the gradients appear from right to left, one per tensor, each with the same shape as its tensor: 1×1 at the output, then 4×1 , 2×2 and 4×4 . The last frame shows where the two weight gradients come from.

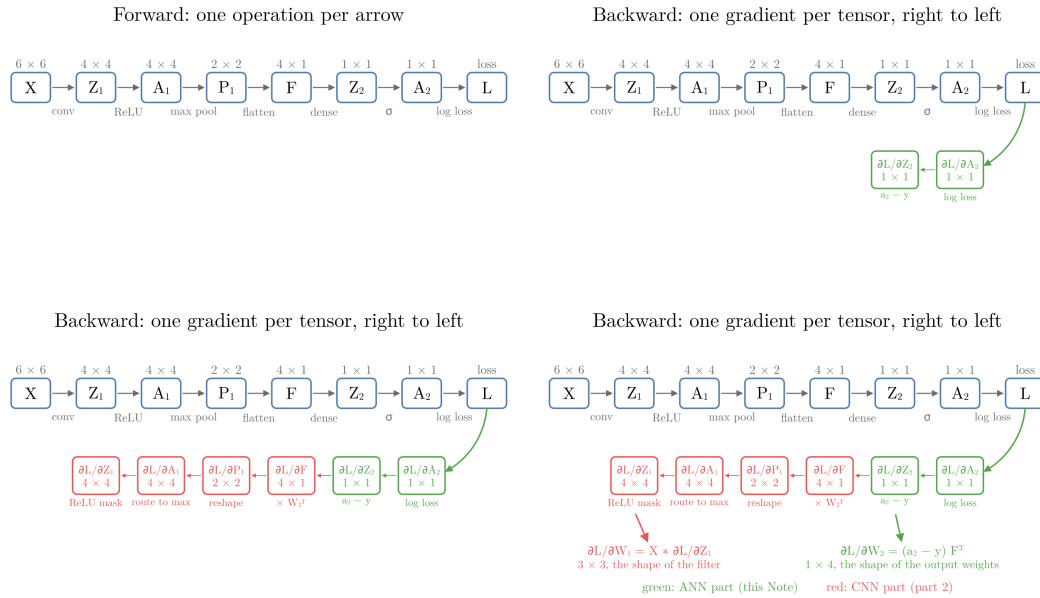


Figure 4: The chain of the small CNN, then the backward pass over it. Each lower box is the gradient of the loss with respect to the tensor above it, with its shape; the word under it is the rule that produces it. Green: the ANN part, found in this Note. Red: the CNN part, found in the part 2 Note

Three of these factors are new: $\partial F / \partial P_1$ goes back through flatten, $\partial P_1 / \partial A_1$ through max

pooling, and $\partial Z_1/\partial W_1$ through the convolution. The part 2 Note finds them.

6. The ANN part: $\partial L/\partial W_2$ and $\partial L/\partial b_2$

Key point: The log loss and the sigmoid cancel: $\partial L/\partial Z_2 = a_2 - y$. Times $F^\top$ for the weights, times 1 for the bias.

6.1 The first two factors

Key point: $\frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} = \frac{a_2 - y}{a_2(1 - a_2)} \cdot a_2(1 - a_2) = a_2 - y$.

Take one image, so A_2 is a single number a_2 . The two factors shared by both chains are exactly those of section 7.1 of the [backpropagation how Note](#):

- differentiating the log loss: $\frac{\partial L}{\partial a_2} = -\frac{y}{a_2} + \frac{1 - y}{1 - a_2} = \frac{a_2 - y}{a_2(1 - a_2)}$;
- the sigmoid's derivative (see the [sigmoid derivative Note](#)): $\frac{\partial a_2}{\partial Z_2} = \sigma(Z_2)(1 - \sigma(Z_2)) = a_2(1 - a_2)$.

Multiplied, $a_2(1 - a_2)$ cancels:

$$\frac{\partial L}{\partial Z_2} = a_2 - y$$

6.2 The last factors

Key point: $Z_2 = W_2 F + b_2$, so $\partial Z_2/\partial W_2 = F$ and $\partial Z_2/\partial b_2 = 1$.

From $Z_2 = W_2 F + b_2 = w_1 f_1 + w_2 f_2 + w_3 f_3 + w_4 f_4 + b_2$, the derivative with respect to each weight w_k is the input f_k it multiplies, and with respect to b_2 it is 1.

1. **In words:** the error at the output, $a_2 - y$, times the input each weight multiplied.
2. **Formula:**

$$\frac{\partial L}{\partial W_2} = (a_2 - y) F^\top, \quad \frac{\partial L}{\partial b_2} = a_2 - y$$

3. **Example:** in the Notebook, the first image is a 0 ($y = 0$) and the untrained network predicts $a_2 = 0.2104$, so $a_2 - y = 0.2104$. Its flattened pooled values F are $(0.4947, 0.4006, 0.5311, 0.7707)$, which gives

$$\frac{\partial L}{\partial W_2} = 0.2104 \times (0.4947, 0.4006, 0.5311, 0.7707) = (0.1041, 0.0843, 0.1117, 0.1621), \quad \frac{\partial L}{\partial b_2} = 0.2104$$

TensorFlow's `GradientTape` (G-88), which differentiates the same network automatically, gives the same numbers (largest difference 0, Notebook).

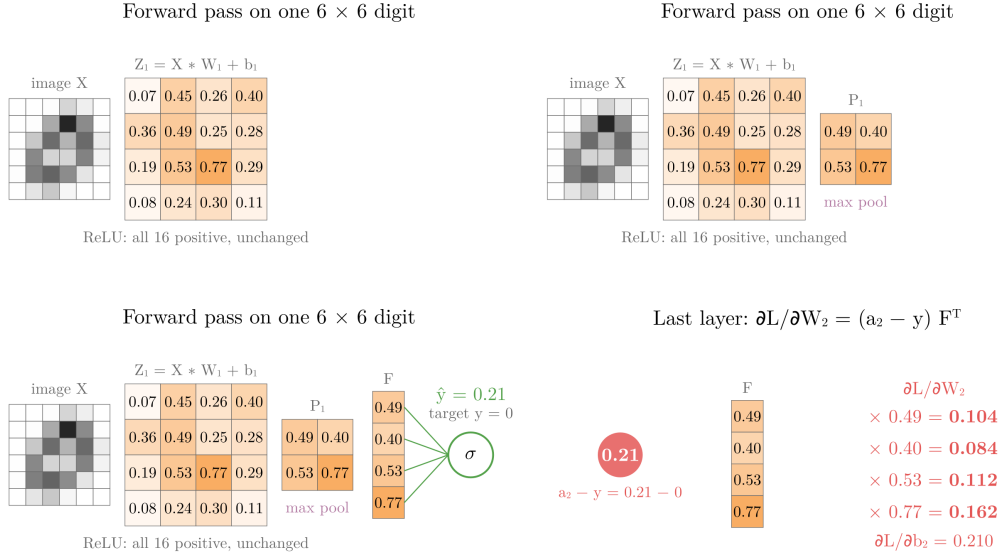


Figure 5: The Notebook's first image, a 0 shrunk to 6×6 , through the small CNN with the Notebook's starting weights. The 3×3 filter slides over X to fill Z_1 ; ReLU changes nothing here, since all 16 values are positive; each 2×2 window sends its maximum to P_1 ; flatten gives F and the sigmoid gives $\hat{y} = 0.21$. Then the error $a_2 - y = 0.21$ multiplies each value of F to give $\partial L / \partial W_2$

Figure 5 plays the forward equations of section 4 on this image, then this section's gradient. Watch the last step: every weight of W_2 gets the same error, 0.21 , times the value of F it multiplied.

6.3 Checking the shapes

Key point: $(1 \times 1)(1 \times 4) = 1 \times 4$, the shape of W_2 . The transpose is what makes the shapes fit.

A derivative is used to update its parameter, so it must have the same shape. W_2 is 1×4 . The error $a_2 - y$ is 1×1 and F is 4×1 , so we use its transpose F^T , 1×4 :

$$\underbrace{(a_2 - y)}_{1 \times 1} \underbrace{F^T}_{1 \times 4} = \underbrace{\frac{\partial L}{\partial W_2}}_{1 \times 4}$$

7. A batch of images

Key point: For m images, F is $4 \times m$ and A_2, Y are $1 \times m$. Then $\partial L / \partial W_2 = \frac{1}{m}(A_2 - Y)F^T$, still 1×4 .

With **mini-batch gradient descent** (G-1222), a batch of, say, 32 or 64 images goes forward together and backpropagation runs once for the batch (see the [gradient descent in neural networks Note](#)).

With m images, each column holds one image:

- F has shape $4 \times m$;
 - the predictions A_2 and the targets Y have shape $1 \times m$.
1. **In words:** the same formula, with the matrices of the whole batch. The loss of the batch is the average of the single losses, so a factor $1/m$ appears.
 2. **Formula:**

$$\frac{\partial L}{\partial W_2} = \frac{1}{m} (A_2 - Y) F^T, \quad \frac{\partial L}{\partial b_2} = \frac{1}{m} \sum_{i=1}^m (a_{2,i} - y_i)$$
 3. **Example (shapes):** $(1 \times m)(m \times 4) = 1 \times 4$. The m cancels in the matrix product, so the derivative is 1×4 whatever the batch size, the shape of W_2 . In the Notebook, with $m = 32$, the formula and `GradientTape` agree to within 6×10^{-17} .

The matrix product adds up the 32 single-image gradients, and the $1/m$ turns the sum into an average.

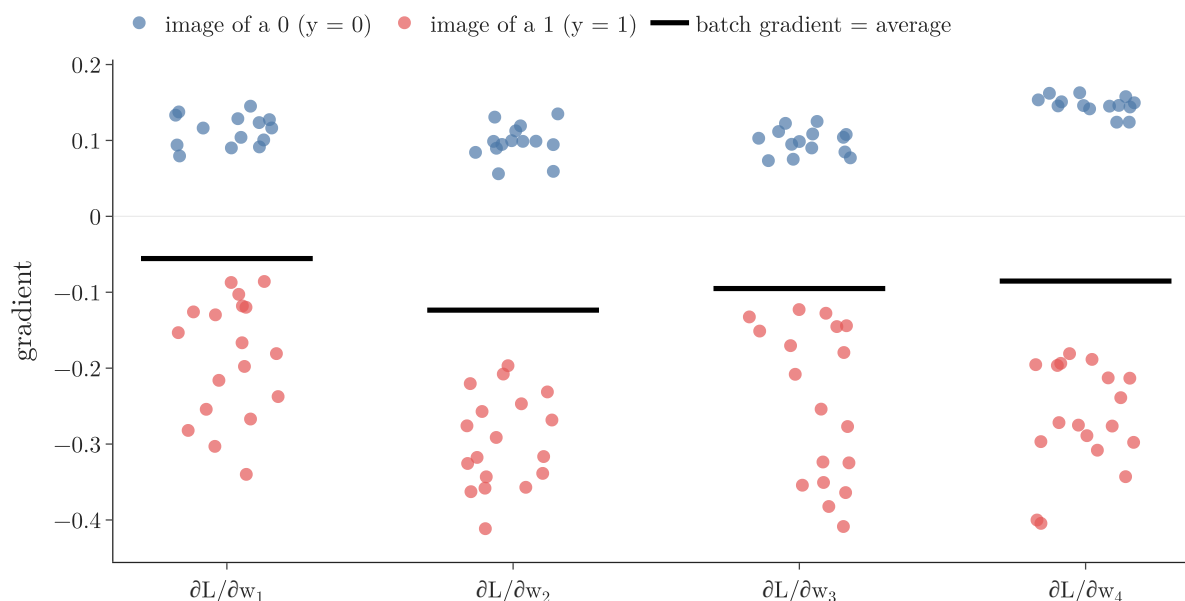


Figure 6: The 32 images of the Notebook’s batch with the starting weights. Each dot is one image’s gradient for one weight of W_2 ; the black bar is the batch gradient, $\frac{1}{m}(A_2 - Y)F^T$, their average.

In Figure 6, the images of 0s (blue) push every weight one way and the images of 1s (red) the other way, because $a_2 - y$ is positive for a 0 and negative for a 1; the batch gradient is the average of both groups.

8. Summary

Derivative	Formula (one image)	Formula (batch of m)	Shape
$\partial L / \partial Z_2$	$a_2 - y$	$A_2 - Y$	1×1 or $1 \times m$
$\partial L / \partial W_2$	$(a_2 - y)F^\top$	$\frac{1}{m}(A_2 - Y)F^\top$	1×4
$\partial L / \partial b_2$	$a_2 - y$	mean of $A_2 - Y$	1×1
$\partial L / \partial W_1, \partial L / \partial b_1$	chains of 7 factors through flatten, max pooling, ReLU and convolution		$3 \times 3, 1 \times 1$

- A CNN is trained like an ANN: forward pass, loss, chain rule, gradient descent.
- The small CNN has 15 parameters: W_1 (9), b_1 , W_2 (4), b_2 .
- Split it into a CNN part and an ANN part; the ANN part gives $\partial L / \partial W_2 = (a_2 - y)F^\top$.
- Every derivative has the shape of its parameter; shapes guide where to put transposes.
- The filter's gradients need the backward steps through flatten, max pooling and convolution: the [part 2 Note](#).

9. Sources

Built from

- CampusX, “Backpropagation in CNN | Part 1 | Deep Learning”, YouTube, <https://www.youtube.com/watch?v=RvCCFttGFMY>

Other references

- TensorFlow API documentation: `tf.GradientTape`, [tensorflow.org/api_docs/python/tf/GradientTape](https://www.tensorflow.org/api_docs/python/tf/GradientTape)

10. Key terms

Term	Meaning
Logical diagram	A drawing of a network as a chain of tensors linked by operations
Forward propagation	Computing the prediction from the input, operation by operation
Trainable parameter	A number changed by gradient descent: filter values, weights and biases
Binary cross-entropy	The log loss $-y \log a - (1 - y) \log(1 - a)$ used for two classes
$\partial L / \partial Z_2 = a_2 - y$	The error at a sigmoid output with log loss
GradientTape	TensorFlow's tool that records a computation and returns its gradients automatically